

Racket - Predavanje 9

Uvod u Racket

1. Sta je Racket?

Racket je funkcionalni programski jezik iz porodice **Lisp/Scheme**. Za razliku od OCaml-a koji koristi staticku provjeru tipova, Racket koristi **dinamicku provjeru tipova** — tipovi se provjeravaju u toku izvršavanja programa, ne pri kompajliranju.

Svaki Racket fajl pocinje sa:

```
#lang racket
```

2. Sintaksa — sve su zagrade!

U Racket-u, **svaki izraz** je oblika (operator operand1 operand2 ...) — ovo se zove **prefiks notacija**.

```
(+ 3 5)           ; = 8      (u OCaml-u: 3 + 5)
(* 2 (+ 3 4))     ; = 14     (u OCaml-u: 2 * (3 + 4))
(> 5 2)           ; = #t     (u OCaml-u: 5 > 2 = true)
(string-append "hello" " " "world") ; = "hello world"
```

Kljucne razlike od OCaml-a:

- #t i #f umjesto true i false
 - ; oznacava komentar (kao (* ... *)) u OCaml-u)
 - Nema razlike izmedju operatora za int i float (+ radi za oba)
-

3. Definisanje varijabli i funkcija

Varijable:

```
(define x 5)
(define y (+ x 3)) ; y = 8
```

Funkcije:

```
(define (kvadrat x) (* x x))
(kvadrat 4)           ; = 16

(define (max2 x y) (if (> x y) x y))
(max2 10 7)           ; = 10
```

U OCaml-u bi ovo bilo:

```
let kvadrat x = x * x;;
let max2 x y = if x > y then x else y;;
```

4. If izraz

```
(if uslov onda_izraz inace_izraz)

(if (> 5 2) "vece" "manje")      ; = "vece"
(if (= 3 3) 1 0)                  ; = 1
```

Vazno: `if` u Racket-u, kao i u OCaml-u, je **izraz** — vraća vrijednost. Razlika je što `if` u Racket-u evaluira **samo jednu granu** (lazy evaluation grana), dok argumenti funkcija se evaluiraju **odmah** (eager evaluation). Ovo će biti ključno za razumijevanje thunks-a.

5. Cond — višestruko grananje

`cond` je Racket-ov ekvivalent `match` ili niza `if-else`:

```
(define (ocjena bodovi)
  (cond
    [(>= bodovi 90) "A"]
    [(>= bodovi 80) "B"]
    [(>= bodovi 70) "C"]
    [#t "F"]))      ; #t je "default" slucaj (kao _ u OCaml-u)

(ocjena 85)         ; = "B"
```

6. Liste

U Racket-u, liste se prave pomoću `list`, `cons`, ili navodnika:

```
(list 1 2 3)          ; = '(1 2 3)
(cons 1 (list 2 3))   ; = '(1 2 3)   (kao 1 :: [2; 3] u OCaml-u)

null                  ; prazna lista   (kao [] u OCaml-u)
(null? (list))        ; = #t           (kao xs = [] u OCaml-u)
```

Pristup elementima:

Racket	OCaml	Opis
<code>(car l)</code>	<code>List.hd l</code>	prvi element (glava)
<code>(cdr l)</code>	<code>List.tl l</code>	ostatak liste (rep)
<code>(null? l)</code>	<code>l = []</code>	provjera da li je prazna

```
(define l (list 10 20 30))
(car l)    ; = 10
(cdr l)    ; = '(20 30)
```

Razlika od OCaml-a: Racket liste mogu sadržavati elemente **razlicitih tipova**:

```
(list 1 "hello" #t (list 2 3)) ; dozvoljeno!
```

U OCaml-u bi ovo bilo nemoguće jer liste moraju biti homogene (`int list`, `string list`, itd.).

7. Provjera tipova u toku izvršavanja

Posto Racket nema statičku provjeru tipova, koristimo **predikate** za ispitivanje tipa:

```
(number? 5)      ; = #t
(number? "hello") ; = #f
(string? "hello") ; = #t
(list? (list 1))  ; = #t
(boolean? #t)     ; = #t
```

8. Anonimne funkcije (lambda)

U OCaml-u koristimo `fun x -> ...`, a u Racket-u `lambda`:

```
(lambda (x) (* x x))          ; anonimna funkcija koja kvadrira
((lambda (x) (* x x)) 5)      ; = 25 (odmah pozivamo)

(define kvadrat (lambda (x) (* x x))) ; isto kao (define (kvadrat x) (* x x))
```

9. Rekurzija

Rekurzija u Racket-u radi isto kao u OCaml-u, ali **ne treba** pisati `rec`:

```
(define (faktorijel n)
  (if (<= n 1)
      1
      (* n (faktorijel (- n 1)))))

(faktorijel 5) ; = 120
```

10. Begin — više izraza u nizu

`begin` omogućava izvršavanje više izraza redom, a vraća vrijednost **zadnjeg**:

```
(begin
  (print "pozdrav")
  (+ 2 3))
; ispisuje "pozdrav", vraća 5
```

Thunks — odloženo izračunavanje

1. Eager vs Lazy evaluacija

U Racket-u (i OCaml-u), argumenti funkcija se evaluiraju **prije** nego što se proslijede funkciji — ovo je **eager (striktna) evaluacija**.

```
(define (moj_if_lose uslov x y) (if uslov x y))
```

Problem: kada pozovemo `(moj_if_lose uslov x y)`, **oba** `x` i `y` ce se evaluirati prije poziva `moj_if_lose`, cak i onaj koji nece biti koristen!

Kod ugradjenog `if`, evaluira se **samo** odabrana grana — `if` je **specijalna forma**, ne obicna funkcija.

2. Sta je thunk?

Thunk je funkcija bez argumenata koja **odlaze izracunavanje** do trenutka poziva.

```
(lambda () izraz) ; thunk – izraz se NE evaluira odmah
```

Pozivanje thunka:

```
(define moj_thunk (lambda () (+ 3 5)))  
(moj_thunk) ; = 8 – tek sada se evaluira (+ 3 5)
```

Ideja: Umjesto da proslijedimo **vrijednost** (koja se odmah izracuna), proslijedimo **thunk** (funkciju koja ce izracunati vrijednost kad je pozovemo). Time kontrolisemo **kada** se izracunavanje desava.

3. Zasto su thunks bitni?

Problem bez thunks-a:

```
(define (moj_if_lose uslov x y) (if uslov x y))  
  
(define (factorial-wrong x)  
  (moj_if_lose (= x 0)  
               1  
               (* x (factorial-wrong (- x 1)))))  
  
; (factorial-wrong 5) – BESKONACNA REKURZIJA!
```

Zasto? Kod poziva `(moj_if_lose (= x 0) 1 (* x (factorial-wrong (- x 1))))`, Racket evaluira **sve argumente** prije poziva funkcije. To znaci da `(* x (factorial-wrong (- x 1)))` se evaluira i kad je `x = 0` — sto vodi u beskonacnu rekurziju!

Rjesenje sa thunks-ovima:

```
(define (moj_if uslov x y) (if uslov (x) (y)))  
  
(moj_if (> 5 2)  
  (lambda () 3)  
  (lambda () (print "evaluacija!")))  
; vraca 3, "evaluacija!" se NE ispisuje
```

Sada `x` i `y` nisu vrijednosti, nego **thunks-ovi** (funkcije). Evaluiramo samo `(x)` ili `(y)` ovisno o uslovu — kao sto ugradjen `if` evaluira samo jednu granu.

4. Thunks vs obicni izrazi

Thunks **nisu uvijek ekvivalentni** obicnim izrazima. Razlika se vidi kod **bocnih efekata** (**side**

effects):

```
(define (f1 e) (list e e e))
(define (f2 e) (list (e) (e) (e)))

(f1 (begin (print "nesto") 5))
; ispisuje "nesto" JEDNOM, vraca '(5 5 5)

(f2 (lambda () (begin (print "nesto") 5)))
; ispisuje "nesto" TRI PUTA, vraca '(5 5 5)
```

Objasnjenje:

- f1 prima **vrijednost** — (begin (print "nesto") 5) se evaluira jednom (pri pozivu), a rezultat 5 se koristi tri puta
 - f2 prima **thunk** — svaki (e) poziva thunk iznova, pa se (print "nesto") izvrši tri puta
-

Zadaci

Zadatak 1 — Izvlacenje brojeva iz ugnijezdene liste

Napisati funkciju koja prima listu čiji elementi mogu biti brojevi, karakteri, ili druga lista istog tipa (npr. lista može biti [1, 'a', ['b', 5, [2, 'c']], 3]), te vraća listu svih brojeva koji su se pojavljivali u prosljedjenoj listi.

Rjesenje: [z1.rkt](#)

```
#lang racket
(define (f l) (cond
  [(null? l) null]
  [(number? (car l)) (cons (car l) (f (cdr l)))]
  [(list? (car l)) (append (f (car l)) (f (cdr l)))]
  [#t (f (cdr l))])
))

(define l (list 1 "a" (list "b" 5 (list 2 "c")) 3))
(f l)      ; = '(1 5 2 3)
```

Ideja: Koristimo cond za provjeru tri slučaja za svaki element (car l):

- Ako je **broj** (number?) — dodajemo ga u rezultat sa cons i nastavljamo rekurzivno
- Ako je **lista** (list?) — rekurzivno izvlačimo brojeve iz te podliste i spajamo sa append
- **Inace** (karakter, string, itd.) — preskakemo ga i nastavljamo sa (cdr l)

Bazni slučaj: prazna lista vraća null.

Zadatak 2 — Razlika između if i moj_if

Dati primjer izraza uslov, x, y gdje poziv (if uslov x y) nije ekvivalentan pozivu (moj_if uslov x y), pri čemu je moj_if funkcija definisana sa

(define (moj_if uslov x y) (if uslov x y)). Kako mozemo modifikovati funkciju `moj_if` pa da ti pozivi budu uvijek ekvivalentni?

Rjesenje: [z2.rkt](#)

Problem — `moj_if_lose` evaluira oba argumenta:

```
(define (moj_if_lose uslov x y) (if uslov x y))

(if (> 5 2) 3 (print "evaluacija!"))
; vraca 3, NE ispisuje "evaluacija!" – jer if evaluira samo odabranu granu
```

Ali:

```
(moj_if_lose (> 5 2) 3 (print "evaluacija!"))
; ispisuje "evaluacija!" i vraca 3 – jer se svi argumenti evaluiraju PRIJE poziva!
```

Rjesenje sa thunks-ovima:

```
(define (moj_if uslov x y) (if uslov (x) (y)))

(moj_if (> 5 2)
  (lambda () 3)
  (lambda () (print "evaluacija!")))
; vraca 3, NE ispisuje "evaluacija!"
```

Ideja: `moj_if_lose` je obicna funkcija — Racket evaluira sve argumente prije poziva (eager evaluation). Ugradjen `if` je **specijalna forma** koja evaluira samo jednu granu. Rjesenje: proslijediti thunks-ove (lambda bez argumenata) i pozvati samo odabrani thunk unutar funkcije.

Zasto `factorial`-wrong ne radi:

```
(define (factorial-wrong x)
  (moj_if_lose (= x 0) 1 (* x (factorial-wrong (- x 1)))))
; BESKONACNA REKURZIJA – (* x (factorial-wrong ...)) se uvijek evaluira!
```

Zadatak 3 — Thunks nisu uvijek ekvivalentni obicnim izrazima

Dati primjer kada thunks nisu ekvivalentni obicnim izrazima.

Rjesenje: [z3.rkt](#)

```
(define (f1 e) (list e e e))
(define (f2 e) (list (e) (e) (e)))

(f1 (begin (print "nesto") 5))
; ispisuje "nesto" jednom, vraca '(5 5 5)

(f2 (lambda () (begin (print "nesto") 5)))
; ispisuje "nestonestonesto" (tri puta), vraca '(5 5 5)
```

Ideja: Kod `f1`, argument `(begin (print "nesto") 5)` se evaluira **jednom** (pri pozivu funkcije), a rezultat 5 se koristi tri puta. Kod `f2`, argument je **thunk** — svaki poziv `(e)` ponovo izvršava tijelo thunka, uključujući bocni efekat `(print "nesto")`. Krajnji rezultat `'(5 5 5)` je isti, ali **bocni efekti** se razlikuju.

Zaključak: Thunks i obični izrazi **nisu ekvivalentni** kada izraz ima **bocne efekte** (side effects) poput ispisa, modifikacije stanja, itd. Thunk odlaze izracunavanje i može ga ponoviti više puta, dok se obični izraz evaluira tačno jednom.

Zadaci za vježbu

V1. Napisati funkciju `duzina` u Racket-u koja prima listu i vraća broj elemenata.

- Primjer: `(duzina (list 1 2 3 4)) -> 4`

V2. Napisati funkciju `suma` u Racket-u koja prima listu brojeva i vraća njihovu sumu. Koristiti `cond` za provjeru prazne liste.

- Primjer: `(suma (list 1 2 3)) -> 6`

V3. Napisati funkciju `sadrzi?` koja prima listu i element, te vraća `#t` ako lista sadrži taj element.

- Primjer: `(sadrzi? (list 1 2 3) 2) -> #t`

V4. Napisati funkciju `moj-map` koja prima funkciju i listu, te vraća novu listu u kojoj je funkcija primijenjena na svaki element. Ne koristiti ugrađeni `map`.

- Primjer: `(moj-map (lambda (x) (* x x)) (list 1 2 3)) -> '(1 4 9)`

V5. Napisati funkciju `moj-filter` koja prima predikat (funkciju koja vraća `#t` ili `#f`) i listu, te vraća listu elemenata za koje predikat vraća `#t`.

- Primjer: `(moj-filter (lambda (x) (> x 3)) (list 1 5 2 8 3)) -> '(5 8)`

V6. Napisati funkciju `obrni` koja prima listu i vraća obrnutu listu, bez korištenja ugrađene `reverse` funkcije.

- Primjer: `(obrni (list 1 2 3)) -> '(3 2 1)`